

Multiclass_tree

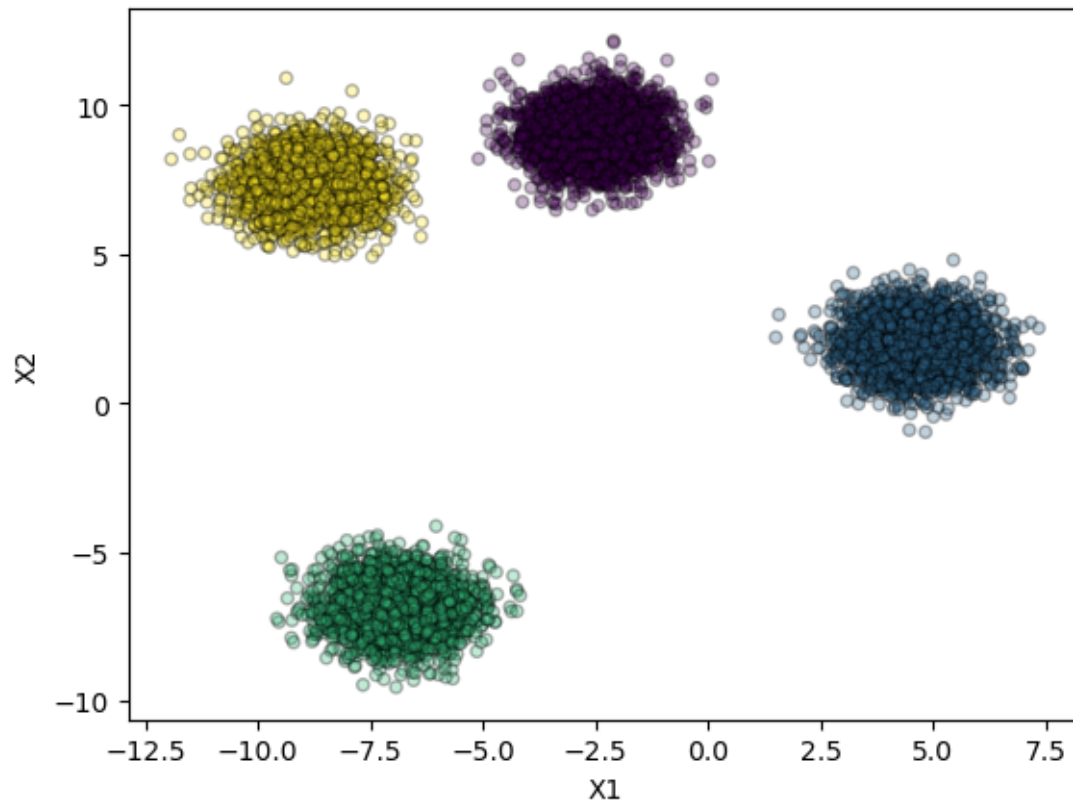
June 7, 2026

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import classification_report
import warnings

warnings.filterwarnings("ignore", category=DeprecationWarning)
```

0.0.1 We'll make our synthetic data

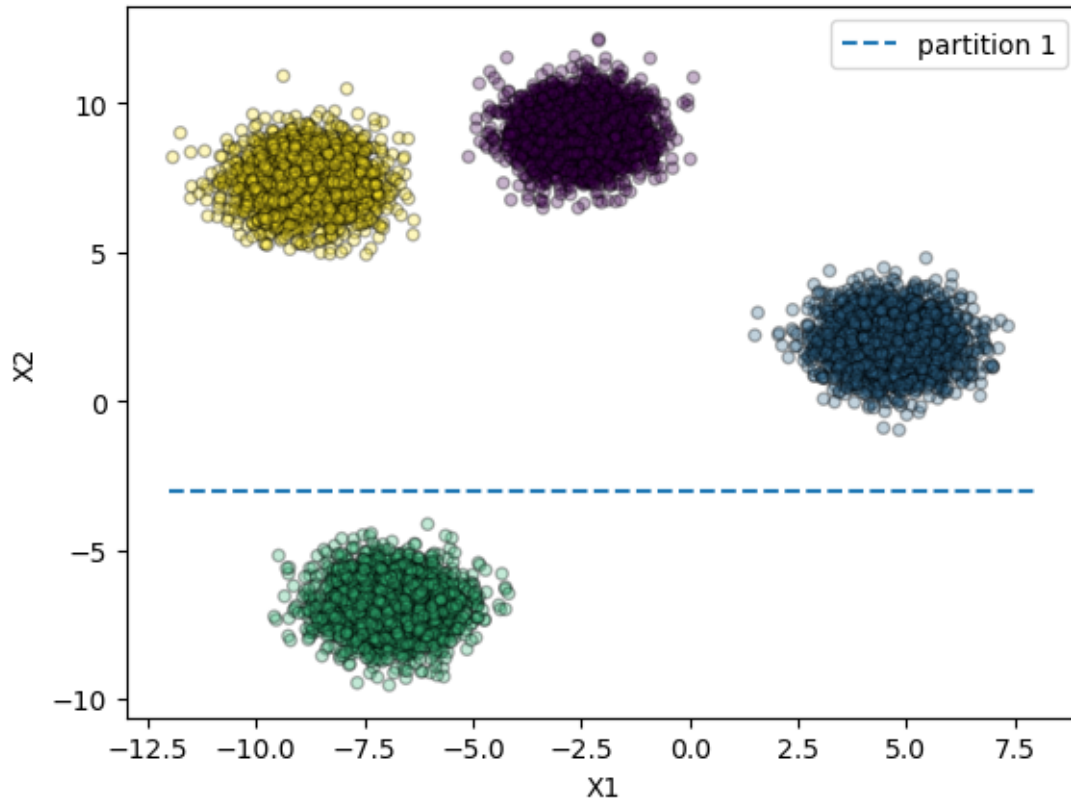
```
[2]: np.random.seed(42)
X, y = make_blobs(n_samples=10000, n_features=2, centers=4, cluster_std=.8)
plt.xlabel('X1')
plt.ylabel('X2')
plt.scatter(X[:, 0], X[:, 1], marker='.', alpha=0.3, ec='k', s=80, c=y)
plt.show()
```



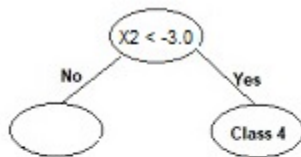
0.0.2 Partition 1

We'll intuitively divide the space recursively. Let's begin splitting the feature space with $X2 < -3.0$

```
[4]: plt.scatter(X[:, 0], X[:, 1], marker='.', alpha=0.3, ec='k', s=80, c=y)
plt.hlines(-3.0, -12., 8., linestyle="dashed", label="partition 1")
plt.xlabel('X1')
plt.ylabel('X2')
plt.legend()
plt.savefig('images/partition1.jpg')
plt.show()
```



We have already isolated Class 4. This gives us the beginning of a tree with one leaf node.

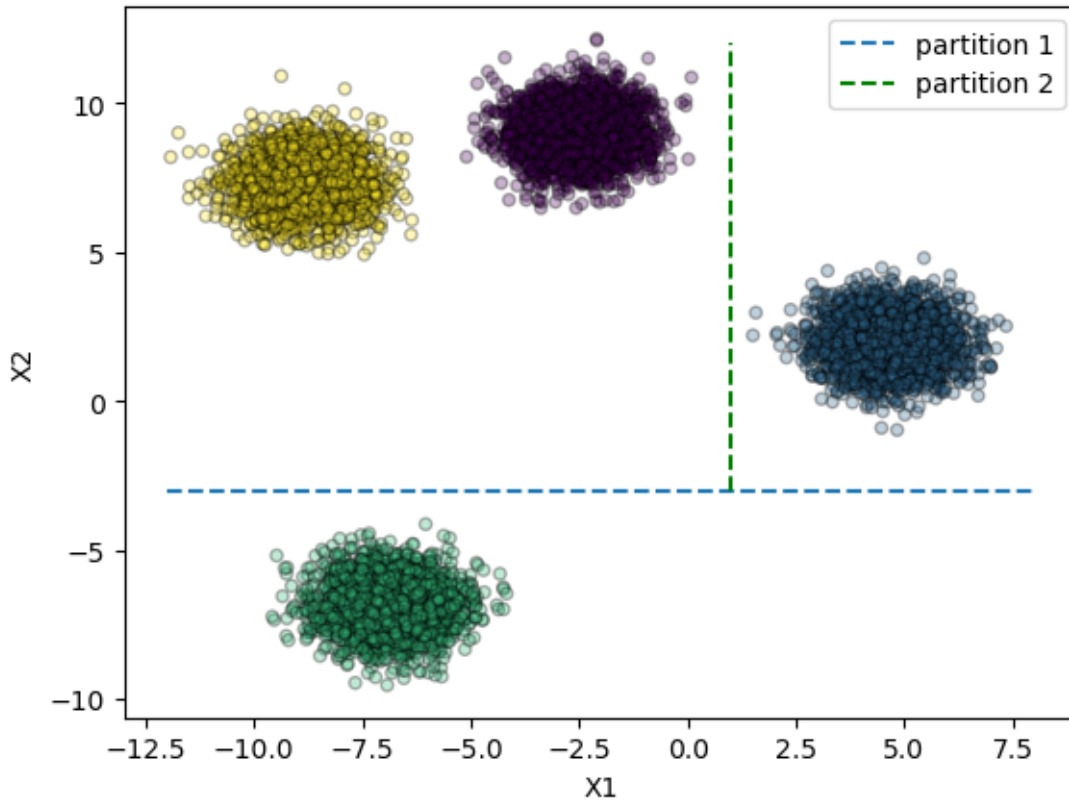


0.0.3 Partition 2

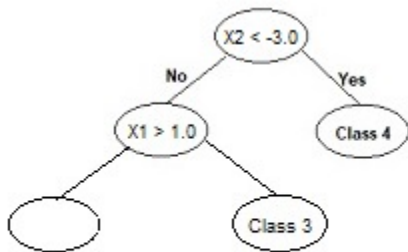
So we have separated the green class (Class 4), leaving Classes 1-3. Now we'll subdivide the top of the space, let's try where $X1 > 1.0$

```
[23]: plt.scatter(X[:, 0], X[:, 1], marker='.', alpha=0.3, ec='k', s=80, c=y)
plt.hlines(-3.0, -12., 8., linestyle="dashed", label='partition 1')
plt.vlines(1., -3.0, 12, linestyle="dashed", colors='green', label='partition_
↪2')
```

```
plt.xlabel('X1')
plt.ylabel('X2')
plt.legend()
plt.savefig('images/partition2.jpg')
plt.show()
```



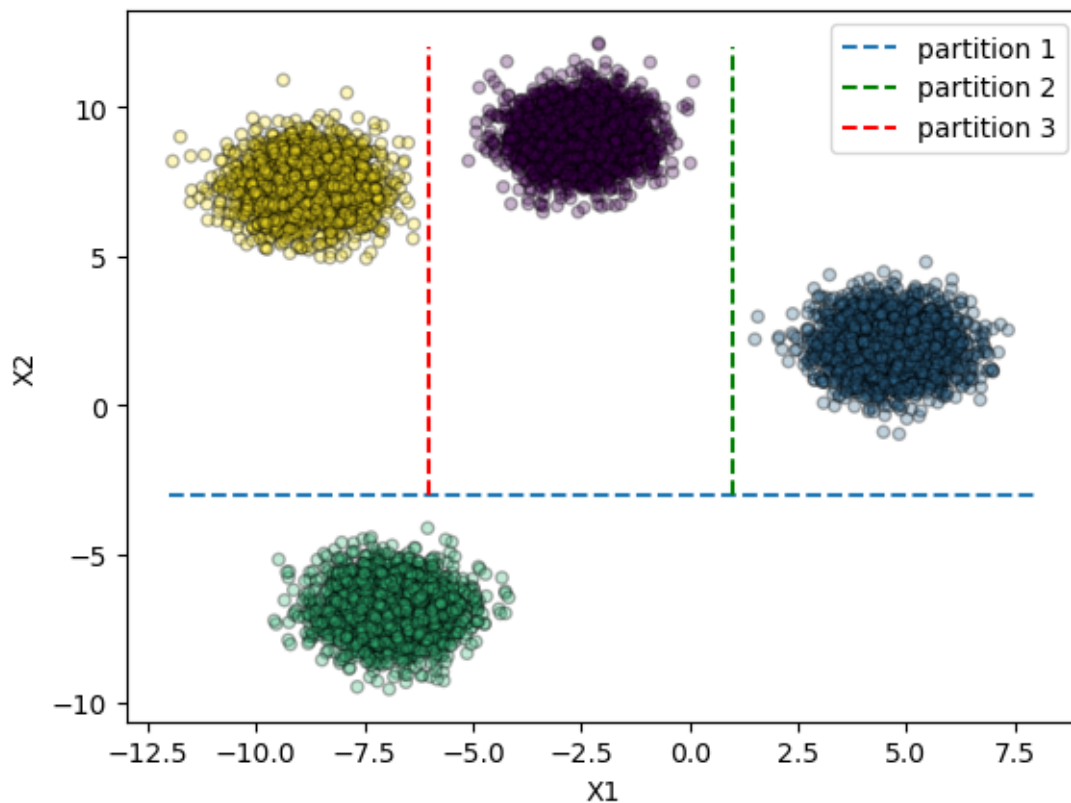
Looks good! We now have isolated Class3. So our tree is now:



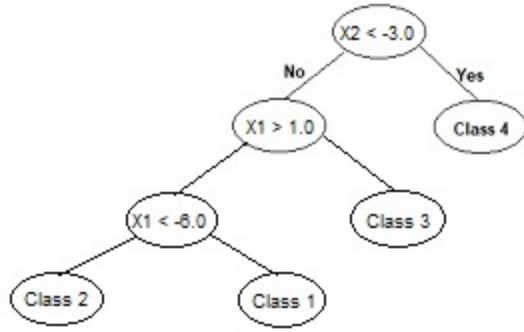
0.0.4 Partition 3

It looks like we can now also separate Classes 1 and 2, with $X1 < -6.0$

```
[24]: plt.scatter(X[:, 0], X[:, 1], marker='.', alpha=0.3, ec='k', s=80, c=y)
plt.hlines(-3.0, -12., 8., linestyle="dashed", label="partition 1")
plt.vlines(1., -3.0, 12, linestyle="dashed", colors='green', label='partition_
↪2')
plt.vlines(-6., -3, 12, linestyle="dashed", color="red", label='partition 3')
plt.xlabel('X1')
plt.ylabel('X2')
plt.legend()
plt.savefig('images/partition3.jpg')
plt.show()
```



Giving our completed Decision Tree as



And we have now reached leaf nodes for each individual Class. With a new example, with the points,

$$X1 = 1, X2 = -2.5$$

We can navigate our tree to predict the class is Class 2, in three steps (the tree's depth).

0.0.5 How the computer does this

Since it can't look at a plot like we can, it has to use an algorithm to recursively partition the feature space.

- 1) Start with the full labeled training dataset at the root node.
- 2) For each node, evaluate all features and possible split points.
- 3) Choose the split that best improves purity (e.g., lowest entropy or Gini Impurity).
- 4) Split the data into child nodes based on that best feature/threshold.
- 5) Repeat steps 2–4 recursively for each child node.
- 6) Stop when a stopping condition is met (max depth, minimum samples per node, or no useful split).
- 7) Assign a prediction to each leaf node (majority class for classification, mean value for regression).
- 8) Optionally prune the tree to reduce overfitting using validation data.

Math At each iteration, we find the best split in one of the features in the partitioned feature set by one of the following metrics. (There are others as well, but we'll just look at these two.)

Entropy

$$H(S) = -\sum_{i=1}^c p_i \log_2(p_i)$$

Where - $H(S)$ is the entropy of the dataset S (or subset) - c total number of classes in the target value - p_i is the probability or proportion of data points belonging to class i

Gini Impurity

$$Gini(S) = 1 - \sum_{i=1}^c (p_i)^2$$

Where - $Gini(S)$ is the Gini impurity of the dataset S - c is the total number of classes - p_i is the probability or proportion of data points belonging to class i

With either, we calculate the metric for each proposed child node, and then we obtain a total of them using this formula:

$$TotalImpurityAfterSplit = (\frac{N_{left}}{N_{parent}} Impurity_{left}) + (\frac{N_{right}}{N_{parent}} Impurity_{right})$$

Where - N represents the number of samples in that specific node

Our aim is to at each split **minimize** the metric. Once we have made the best split, we divide the space according to each node. If that is a leaf node (or reaches our stopping criteria) we no longer use that portion of the data.

0.0.6 Try with sklearn DecisionTree

Start with splitting the data into train and test sets.

Notice that, for decision trees, we **do not** need to scale the data. The tree can handle a range of magnitudes. (However, if you are trying different classifiers, you may want to scale the data for consistency.)

```
[6]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.2,
↳ random_state=42)
```

```
[13]: tree = DecisionTreeClassifier()
tree.fit(X_train, y_train)
```

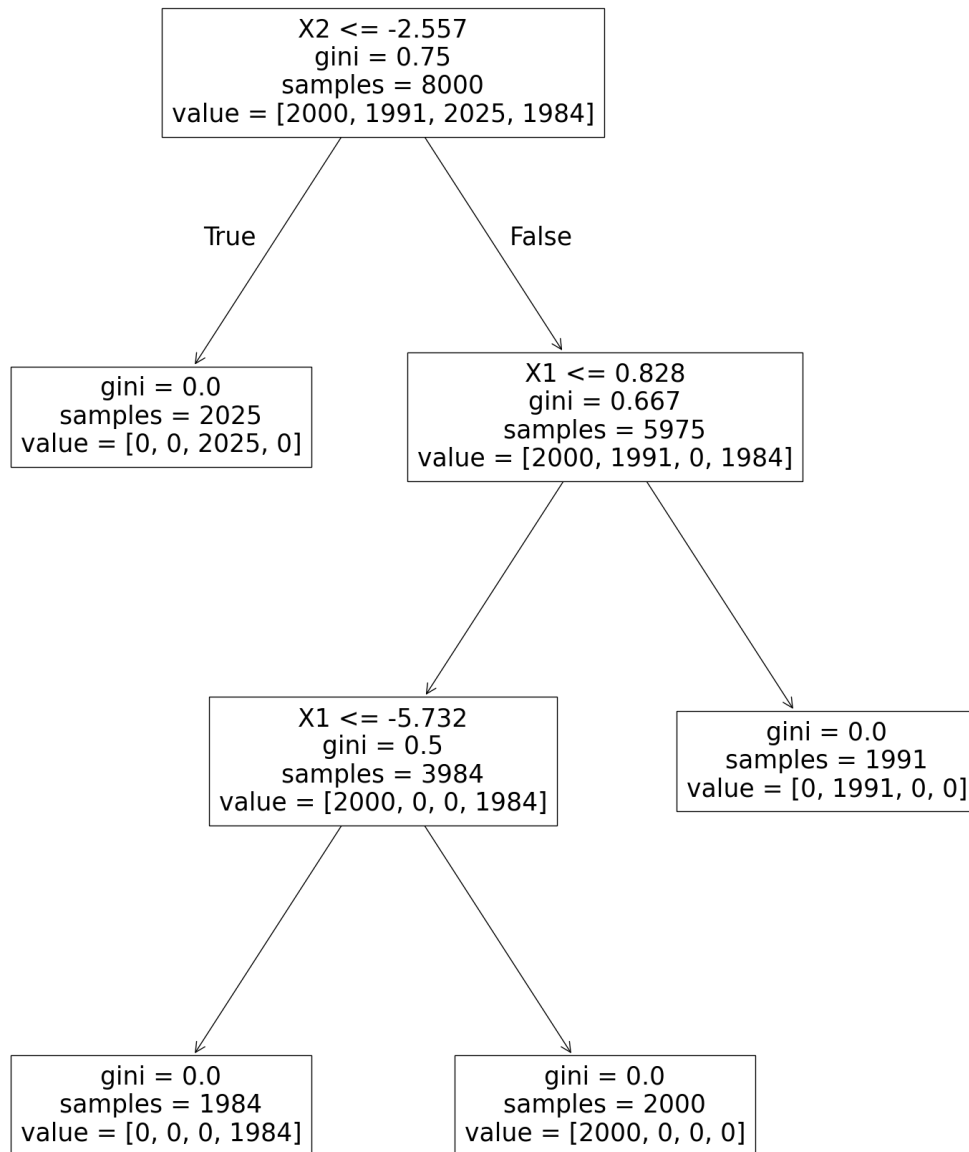
```
[13]: DecisionTreeClassifier()
```

```
[14]: y_hat = tree.predict(X_test)
print(classification_report(y_test, y_hat))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	500
1	1.00	1.00	1.00	509
2	1.00	1.00	1.00	475
3	1.00	1.00	1.00	516
accuracy			1.00	2000
macro avg	1.00	1.00	1.00	2000
weighted avg	1.00	1.00	1.00	2000

We can also view how the DecisionTreeClassifier algorithm divided the space. While it may partition the feature space differently than our manual partition (and some of it is random), it ended up comparable logic.

```
[16]: plt.figure(figsize=(20, 25))
plot_tree(tree, feature_names=['X1', 'X2'])
plt.show()
```



Again, great result, but this is simple problem.